

Продвинутая работа со списками

Функция enumerate() — Индексация в Циклах

Функция enumerate() является одним из самых полезных инструментов в Python для работы со списками, кортежами и другими итерируемыми объектами.

Что делает enumerate()?

В стандартном цикле for вы получаете только сам элемент:

```
data = ['A', 'B', 'C']
for item in data:
    # У нас есть 'A', 'B', 'C', но нет их индексов (0, 1, 2)
    print(item)
```

Функция enumerate() решает эту проблему. Она принимает итерируемый объект и возвращает пары из индекса и значения для каждого элемента.

enumerate(список) -> (индекс, значение)

Основной Синтаксис

Для использования enumerate() в цикле for необходимо указать две переменные: одну для индекса, другую для значения.

```
# Синтаксис: for индекс, значение in enumerate(список):
items = ['Яблоко', 'Банан', 'Вишня']

for index, fruit in enumerate(items):
    print(f"Индекс: {index}, Фрукт: {fruit}")
```

Вывод:

Индекс: 0, Фрукт: Яблоко

Индекс: 1, Фрукт: Банан

Индекс: 2, Фрукт: Вишня



Пример 1: Изменение элемента по индексу

enumerate() часто используется, когда нужно изменить элемент в списке на месте, так как мы получаем точный индекс.

```
scores = [80, 95, 70, 60]
```

```
# Добавляем 5 баллов к результатам, которые меньше 85
for i, score in enumerate(scores):
    if score < 85:
        # Используем индекс i для изменения исходного списка
        scores[i] = score + 5

print(f"Измененные баллы: {scores}")
# Вывод: [85, 100, 75, 65]
```

⚙ Неobligательный аргумент start

По умолчанию индексация начинается с 0. Вы можете изменить начальное значение, используя аргумент start.

```
tasks = ['Собрать данные', 'Анализ', 'Отчет']

# Начинаем нумерацию с 1
for num, task in enumerate(tasks, start=1):
    print(f"Задача №{num}: {task}")
```

Вывод:

Задача №1: Собрать данные

Задача №2: Анализ

Задача №3: Отчет

Работа с Вложенными Списками (Матрицами)

Вложенный список — это список, который содержит другие списки в качестве своих элементов. Вложенные списки чаще всего используются для представления табличных данных, сеток или матриц (двумерных массивов).

Структура Матрицы

Матрица N x M в Python представляется как список, где каждый элемент — это список-строка.

```
# Матрица 3x3 (3 строки, 3 столбца)
matrix = [
    [10, 20, 30], # Строка 0
    [40, 50, 60], # Строка 1
```

```
[70, 80, 90] # Строка 2  
]
```

1. Доступ к элементам

Для доступа к элементу в матрице используется двойная индексация:

```
matrix[индекс\строки][индекс\столбца]
```

```
print(matrix[0])    # Вывод: [10, 20, 30] (Целая строка)  
print(matrix[1][2]) # Вывод: 60 (Элемент в строке 1, столбце 2)
```

2. Использование Вложенных Циклов

Для обработки всех элементов матрицы необходимо использовать вложенные циклы `for` — один для итерации по строкам, а второй для итерации по элементам внутри каждой строки (столбцам).

```
matrix = [  
    ['A', 'B'],  
    ['C', 'D']  
]  
  
# Внешний цикл: итерируется по строкам  
for row in matrix:  
  
    # Внутренний цикл: итерируется по элементам внутри текущей  
    # строки  
    for element in row:  
        print(element, end=' ') # Вывод: A B C D  
  
    print() # Переход на новую строку после обработки всех  
    # элементов
```

💡 Пример 2: Найти сумму всех элементов

```
matrix = [  
    [1, 2],
```

```

    [3, 4]
]
total_sum = 0

for row in matrix:
    for number in row:
        total_sum += number

print(f"Общая сумма: {total_sum}") # Вывод: 10

```



Пример 3: Вложенные циклы с enumerate()

Если вам нужны индексы и строк, и столбцов, вы комбинируете enumerate():

```

grid = [
    ['X', 'O'],
    ['O', 'X']
]

# 1. enumerate() для строк: i (индекс строки), row (сама строка)
for i, row in enumerate(grid):

    # 2. enumerate() для столбцов: j (индекс столбца), cell (значение)
    for j, cell in enumerate(row):

        if cell == 'X':
            print(f"Найдено 'X' в координатах: [{i}, {j}]")

```

Вывод:

Найдено 'X' в координатах: [0, 0]

Найдено 'X' в координатах: [1, 1]